MINI PROJECT
REPORT

NATIONAL INSTITUTE OF TECHNOLOGY
KARNATAKA, SURATHKAL, MANGALORE- 575025



P2PFaaS: A framework for FaaS peer-to-peer scheduling and load balancing in Fog and Edge computing

Guide:- Dr. Sourav Kanti Addya
Submitted By: Bhagwat Poorva Milind

CONTENTS	Pg. No.
1. INTRODUCTION	3
2. OVERVIEW	4
3. TECHNICAL SETUP	5
4. WORK UNDERTAKEN	7
5. IMPLEMENTATION	10
6. RESULTS	14
7. CHALLENGES	21
8. CONCLUSION	24
9. FUTURE WORK	25

INTRODUCTION

The *Peer-to-Peer Function-as-a-Service (P2PFaaS)* platform is designed to provide decentralized serverless computing by enabling peers to execute and schedule functions without a centralized orchestrator. Monitoring and observability are crucial in such distributed systems to understand performance bottlenecks, ensure reliability, and aid in debugging. This document has details of implementing a Metrics Collection System for P2PFaaS, focusing on the Scheduler service.

This phase is aimed at establishing a basic observability using Prometheus and Grafana, collecting and visualizing essential runtime metrics. The goal is to validate the setup, build familiarity with the stack, and gather useful real-time insights into core operations like queue handling and job execution. The effort integrates with the broader setup and configuration of the P2PFaaS stack, including environment setup, Docker configurations, service integrations, and a deployed function (pigo-faceblur) to test end-to-end scheduling and metric generation.

OVERVIEW

The metrics collection implementation for P2PFaaS focuses on establishing a Proof of Concept (POC) for core metric instrumentation and visualization. The scope of this phase includes:

- Cloning and configuring the P2PFaaS repositories, particularly focusing on the Scheduler and Discovery components.
- Setting up and running Prometheus for scraping metrics and Grafana for visualizing them.
- Enabling and customizing metrics collection in the Scheduler codebase (metrics.go, records.go, services.go).
- Deploying and validating the metrics pipeline through interactions with the pigo-faceblur function, ensuring end-to-end system visibility.
- Confirming metrics such as queue fill levels, job execution durations, and invocation counts are correctly exposed and visualized.

Key deliverables from this phase include:

- A functional Prometheus and Grafana setup.
- A working /metrics endpoint on the Scheduler.
- Dashboards for real-time system visibility.
- Successful test invocations with confirmed metric flow.

TECHNICAL SETUP

This section describes the tools, technologies, and environment used to implement and test the P2PFaaS framework.

Hardware Requirements:

- Processor: Intel Core i5/i7 or AMD Ryzen 5 (Quad-core or higher)
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: SSD with at least 100 GB free space
- Network: Stable LAN or Wi-Fi connection for P2P communication tests

Software Stack:

- Operating System: Ubuntu 22.04 LTS (Primary Development)
- Programming Language: Python 3.10+

Cloud & Edge Emulation:

- Docker Compose: For simulating distributed nodes (Fog/Edge) on a single machine
- Custom Peer Manager Script: Handles node discovery, task scheduling, and load balancing

Cloning the Required Repositories from:-
https://gitlab.com/p2p-faas

git clone https://gitlab.com/p2p-faas/stack.git

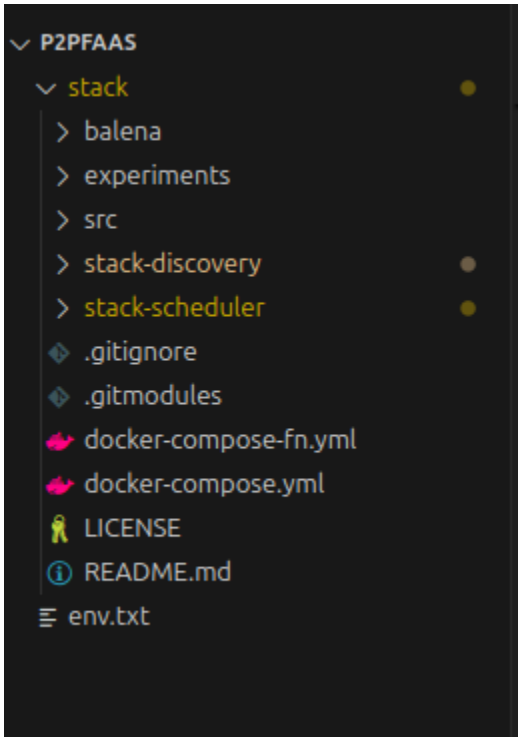
Inside your working directory, clone the repositories in order:-

For cloning balena :- git clone https://gitlab.com/p2p-faas/stack.git

For cloning experiments :- git clone https://gitlab.com/p2p-faas/experiments.git

For cloning stack-discovery :- git clone https://gitlab.com/p2p-faas/stack-discovery.git

For cloning stack-scheduler :- git clone https://gitlab.com/p2p-faas/stack-scheduler.git



WORK UNDERTAKEN

As part of the industrial training project titled "P2PFaaS: A Framework for FaaS Peer-to-Peer Scheduling and Load Balancing in Fog and Edge Computing", I was involved in both the initial system setup and the implementation of a basic metrics collection system to monitor the performance of distributed scheduling services.

Initial Work and System Foundation

Prior to the metrics-related task, I contributed to the initial setup and implementation of the P2PFaaS prototype, which involved:

- Setting up a peer-to-peer environment using Docker Compose to simulate Fog/Edge nodes.
- Developing the Scheduler module responsible for distributing function execution requests across nodes.
- Implementing basic REST APIs for function execution and inter-node communication using Flask.
- Creating a modular and extensible project structure for function execution, peer management, and load balancing.

This groundwork laid the foundation for introducing monitoring and observability capabilities into the system.

Metrics Collection Implementation

The main focus of my current task was to implement a basic metrics collection pipeline to monitor system performance, specifically targeting the Scheduler component. This phase was designed as a Proof of Concept (POC) to validate feasibility and learnings before expanding observability across other services.

Objective:
To set up Prometheus and Grafana for real-time monitoring of Scheduler

performance, and collect essential metrics for visibility, debugging, and future optimization.

Work Breakdown:

- Infrastructure Setup:
    - Deployed Prometheus and Grafana in the development environment.
    - Defined Prometheus scrape configurations for the Scheduler service.
- Metrics Instrumentation:
    - Implemented collection of the following core metrics in the Scheduler:
        - Queue Metrics: Queue size, free slots, and average wait times.
        - Job Execution Metrics: Number of active jobs, execution durations, and invocation counts.
    - Exposed metrics in Prometheus-compatible format via /metrics endpoint using prometheus_client.
- Dashboard and Validation:
    - Verified end-to-end metrics flow from the Scheduler to Prometheus.
    - Created a basic Grafana dashboard for visualizing Scheduler queue behavior and execution stats.
    - Ensured data was being retained appropriately (testing with a 1-week window).

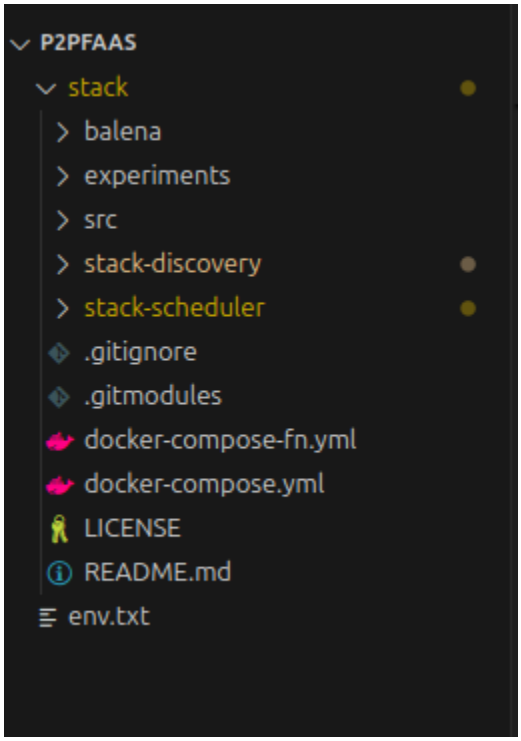
- Operational Considerations:
    - Explored optimal scrape intervals and retention policies to balance overhead and visibility.
    - Documented troubleshooting steps for common issues like missing metrics, port conflicts, and storage growth.

IMPLEMENTATION

This section outlines the step-by-step setup and configuration of the P2PFaaS framework, Scheduler service, Prometheus-based monitoring, and Grafana dashboard. It also covers integration with a sample serverless function for testing, including the face detection use-case (Pigo FaceBlur).

1. Cloning and Organizing Project Repositories

To begin with the setup, the required repositories were cloned from the official P2PFaaS GitLab group:



2. Scheduler Configuration and Environment Setup

After cloning, configuration steps were carried out for the Scheduler and Discovery services.

- Navigated to the stack-scheduler directory and ensured it was the working root for scheduler development.

- Exported essential environment variables for the services using terminal commands.
- Configured constants inside the config.go file of the scheduler to point to local IPs or appropriate gateway addresses.

3. **Scheduler Script Execution and Setup**

Scheduler was configured using scripts from the experiments module:

- Created and activated a Python virtual environment:
- Made configure_discovery.sh executable and ran it

4. **Benchmark Setup**

   Add the URL of OpenFaas UI pigo-faceblur function in the test_req.py file.

```
URL = "http://localhost:18080/function/pigo-faceblur"
```

5. **Navigate to the p2pFaas/stack/stack-scheduler/src/scheduler/faas/**

   Open services.go and take NOT of `config.GetOpenFaasEnabled()`

   function:

```
if !config.GetOpenFaasEnabled()
```

6. **Prometheus Metrics Setup and Configuration**

To enable system observability and performance tracking:

Prometheus Installation and Setup:

- Cloned Prometheus:
- Created prometheus.yml configuration file:
- Launched Prometheus server:

Instrumenting the Scheduler:

- Navigated to the metrics.go file under:

  Ensured metrics service was enabled and logging initialized. Modified records.go:

- Uncommented metric definitions.

- Added imports:
  In services.go, enabled metric tracking logic for job queues, execution times, and request rates.

Verifying Metrics:

Launched the Scheduler:

Verified Prometheus target status at:

Checked exposed metrics:

7. **Sample Function Deployment and Benchmark (Pigo FaceBlur)**

To test end-to-end scheduling and invocation, a face detection function was deployed and invoked using OpenFaaS:

Prepare Input:

- Converted image to Base64:

Function Invocation:

- Navigated to benchmark directory:

  Modified test_req.py to include OpenFaaS gateway:

Invoked the function:

Output:

- Received function execution time and status (e.g., HTTP 200 OK).

8. **Grafana Setup and Dashboard**

   Installing Grafana:

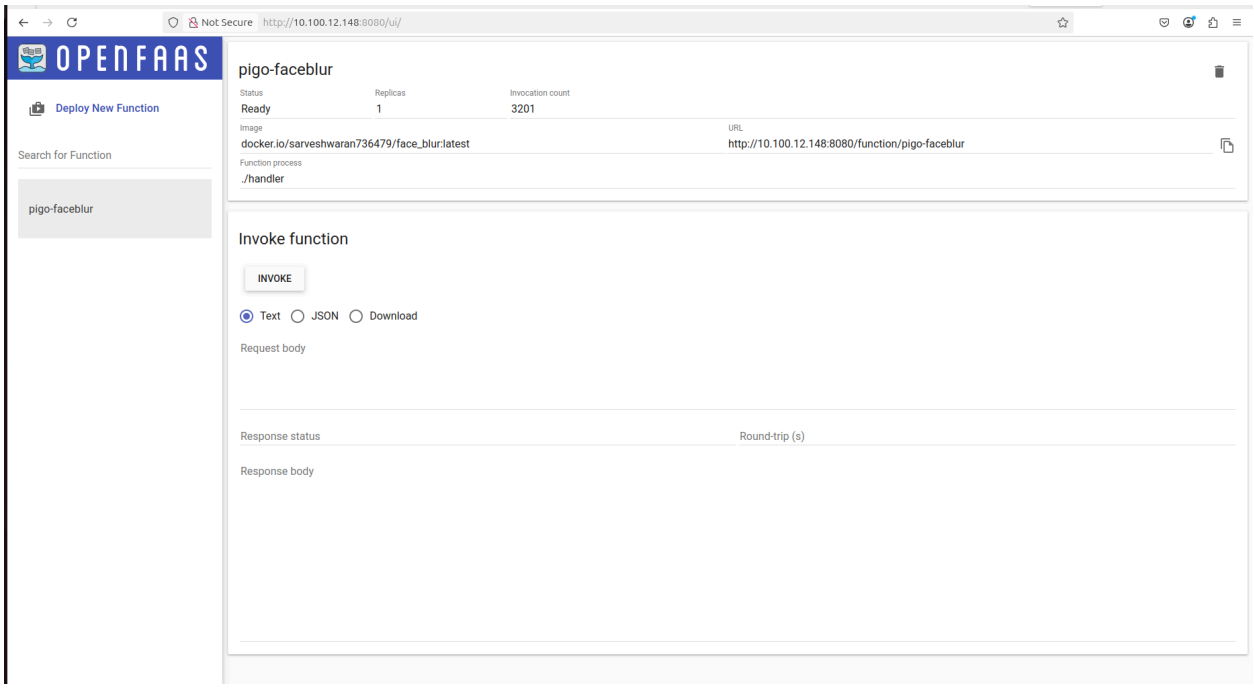
Starting Grafana Server:

Connecting Grafana with Prometheus:

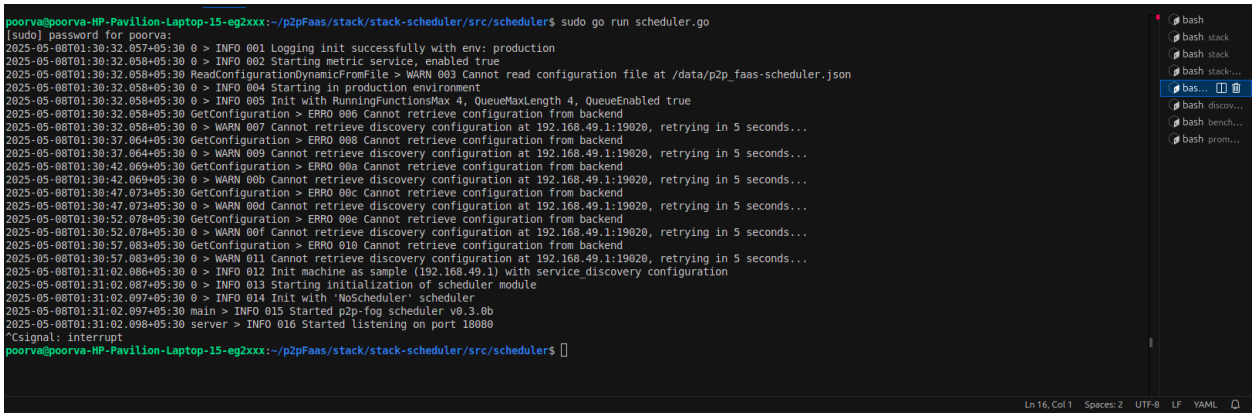
- Accessed Grafana on browser at http://localhost:3000
- Added Prometheus as a data source (http://localhost:9090)
- Created dashboards to visualize metrics:
  - Queue fill level
  - Job execution duration
  - Request rate
  - Function invocation trends

RESULTS

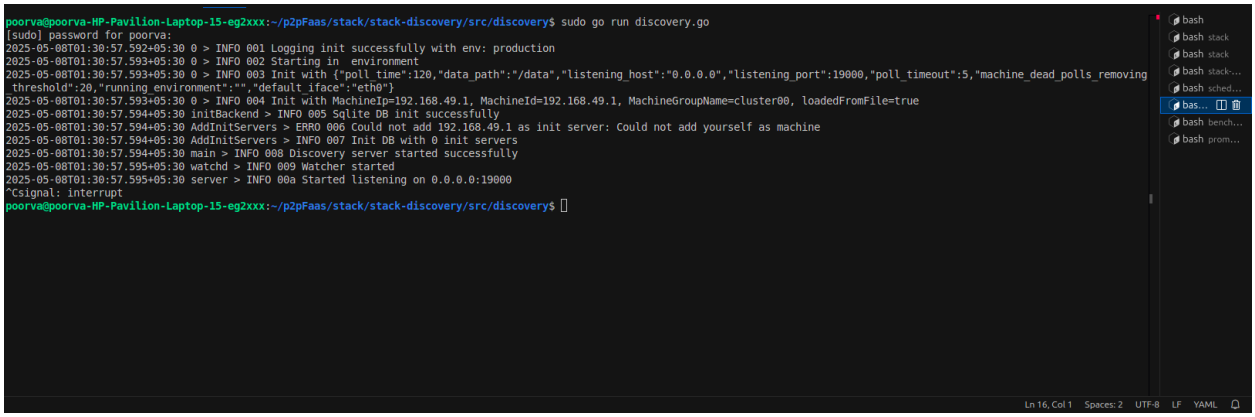
1. OpenFaas looks like this:-



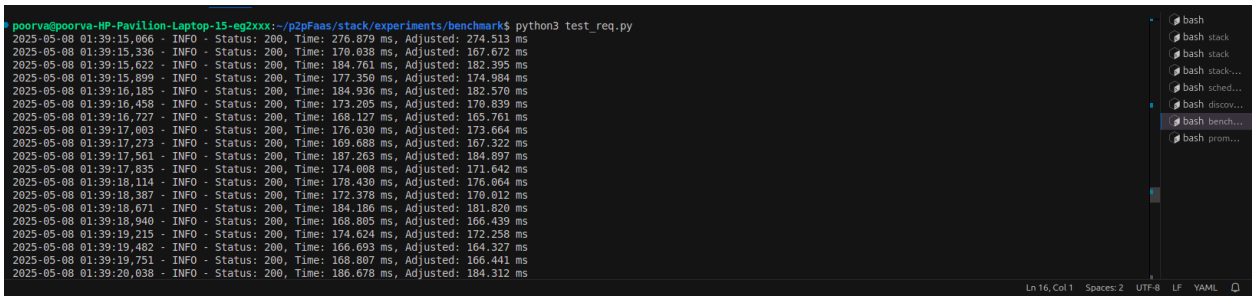
2. When you run scheduler.go



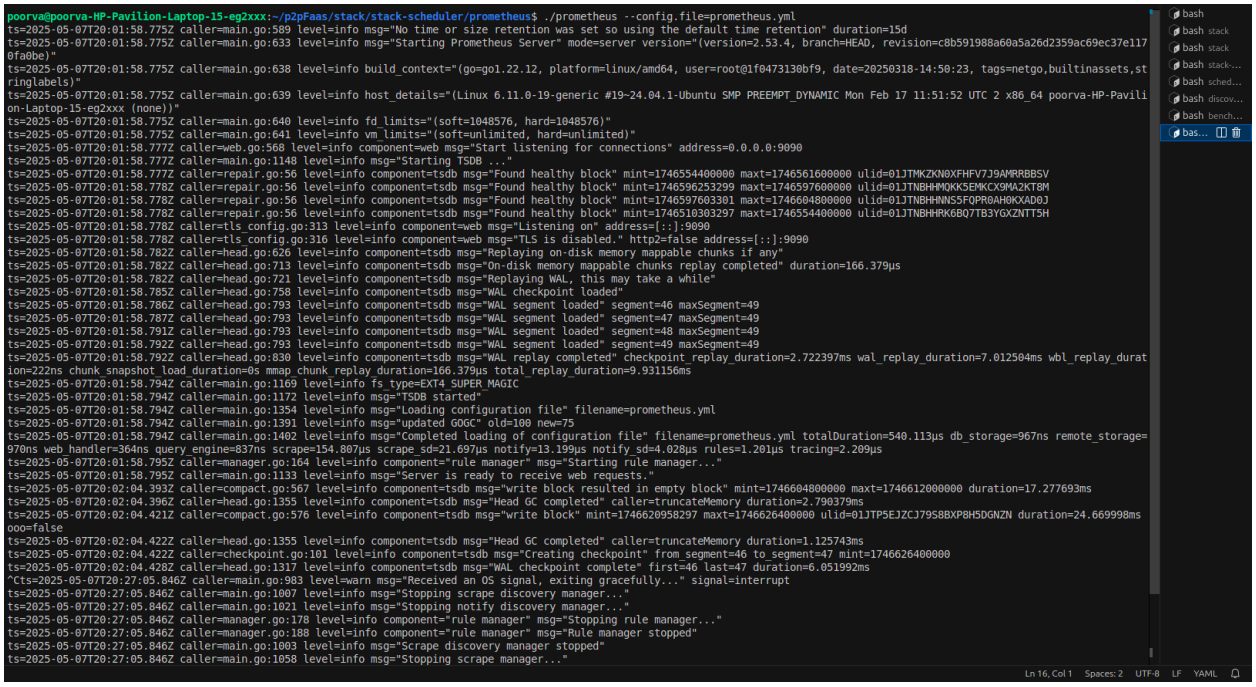
3. After running discovery.go



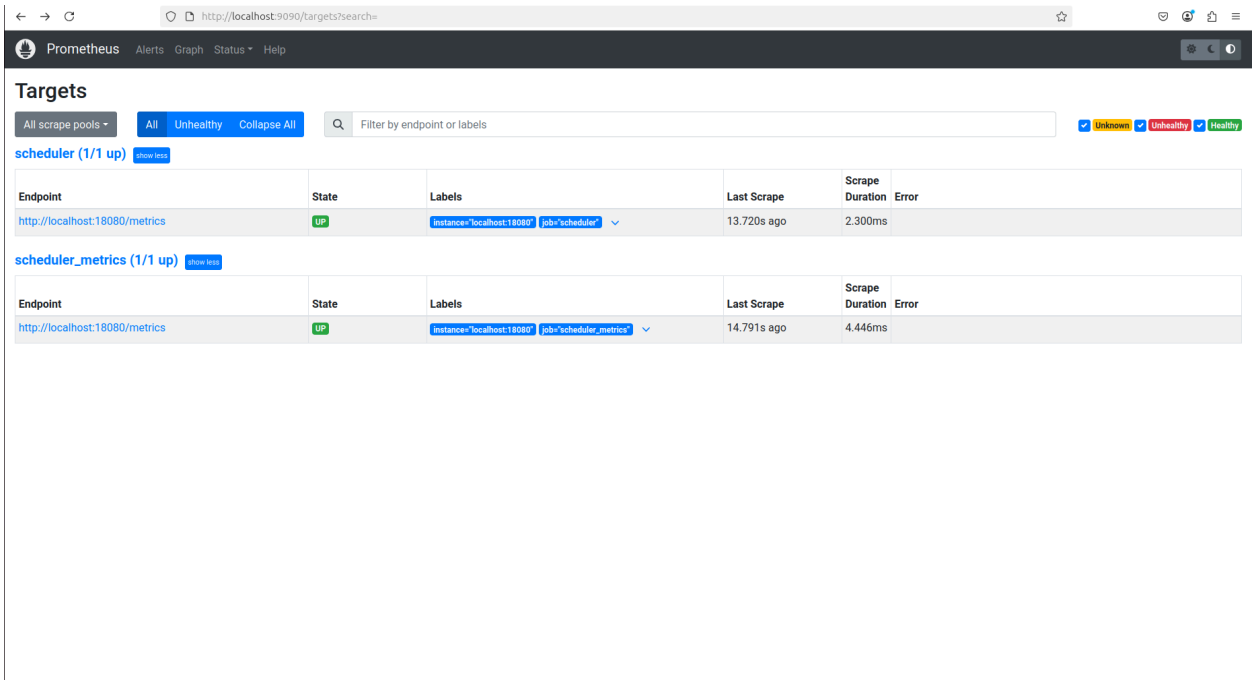
4. After running benchmark test



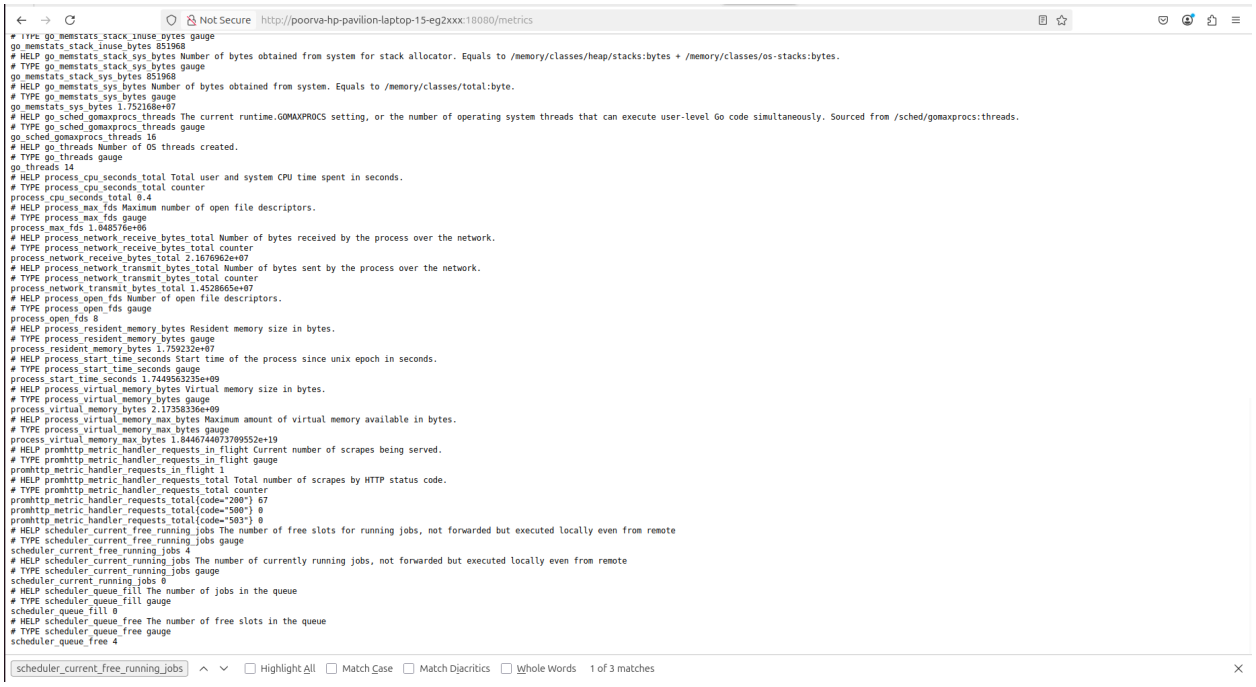
5. After running prometheus.yml



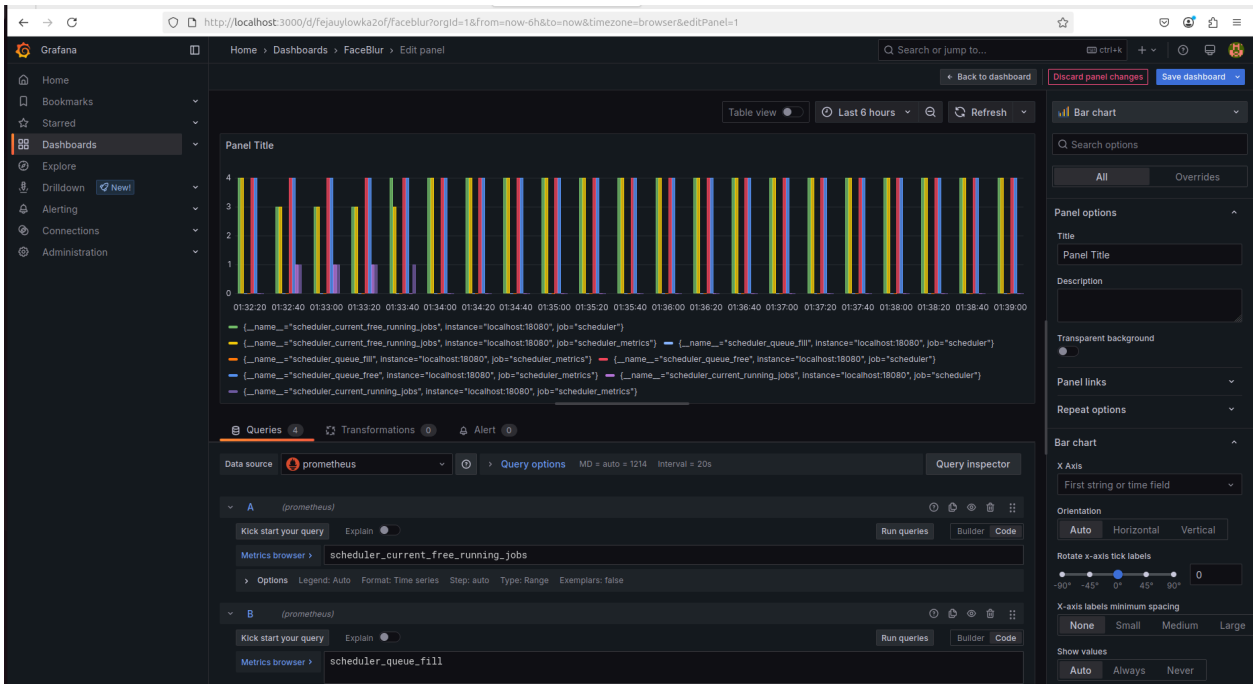
6. Prometheus

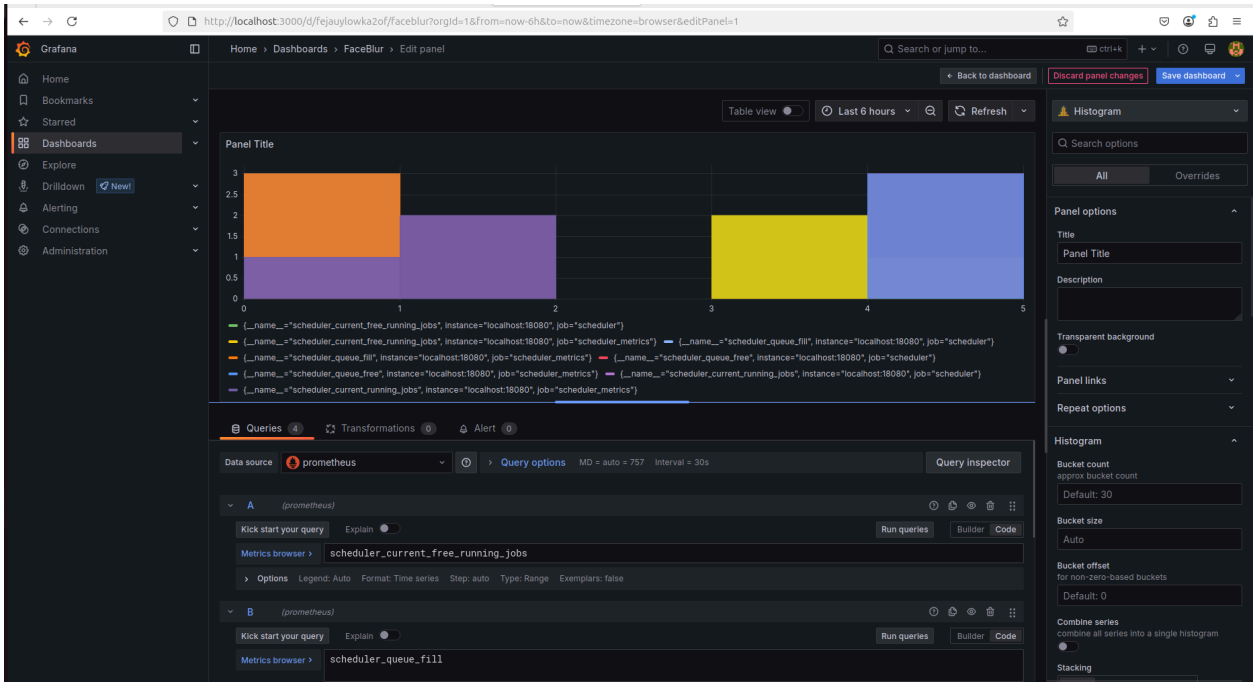


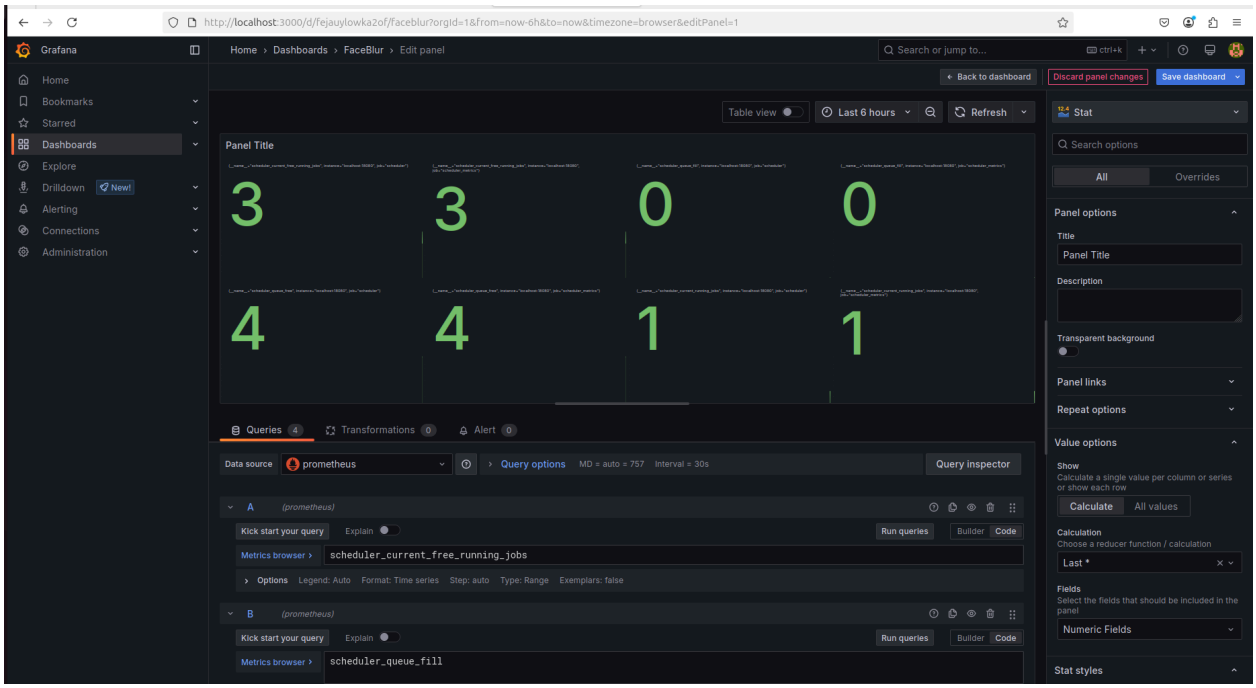
7. Metrics collection

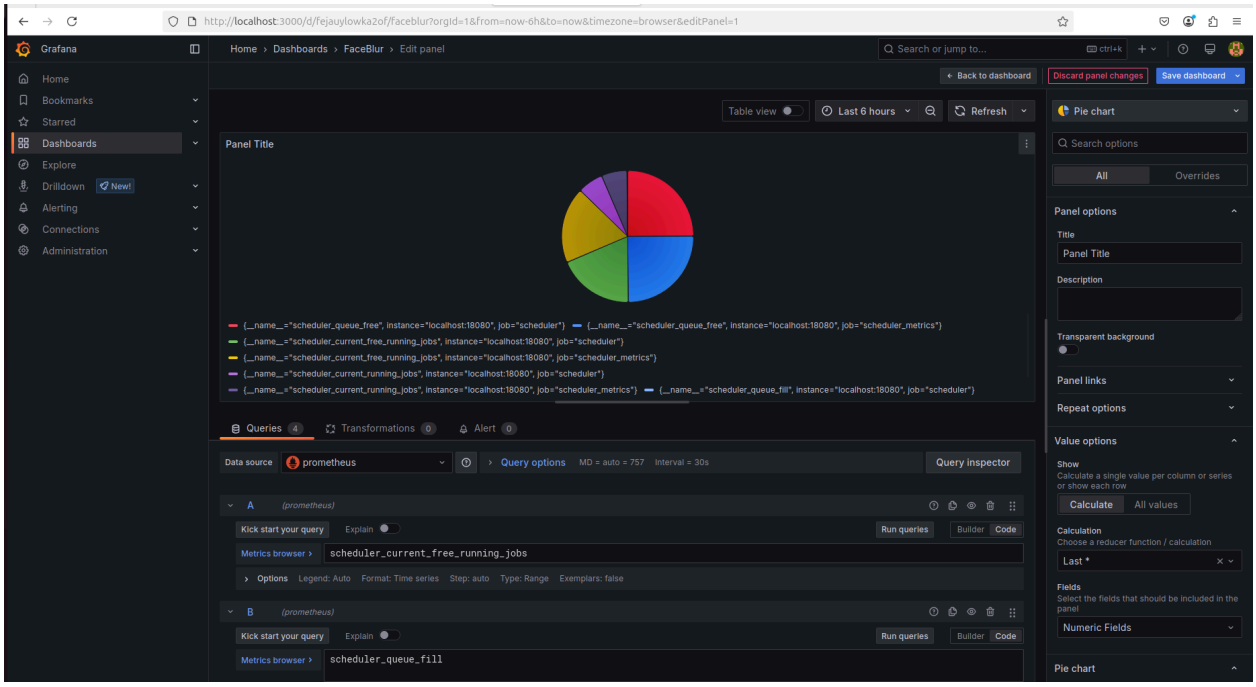


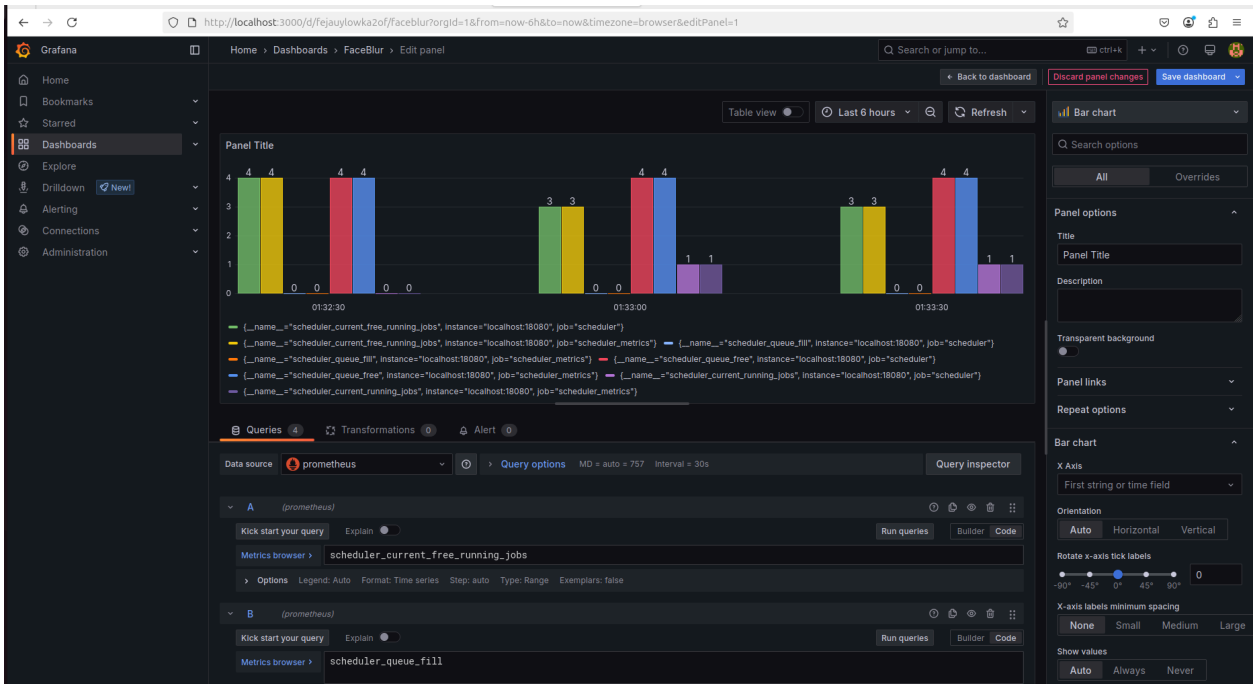
8. Grafana Results

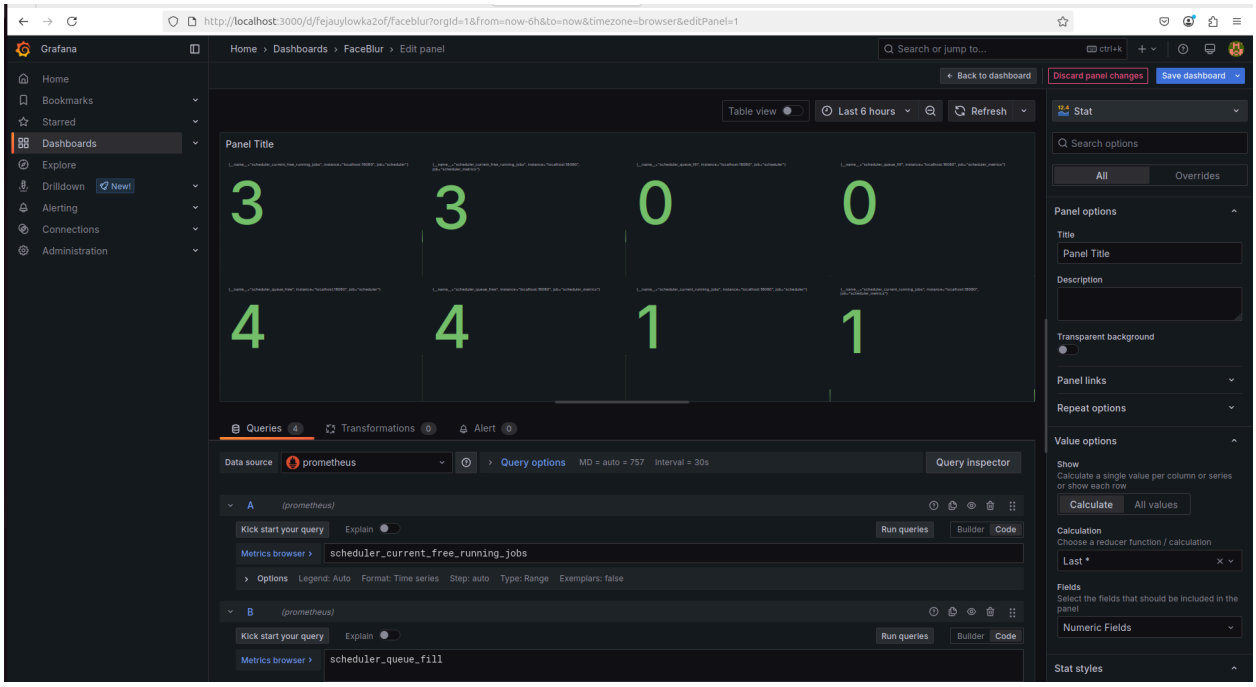


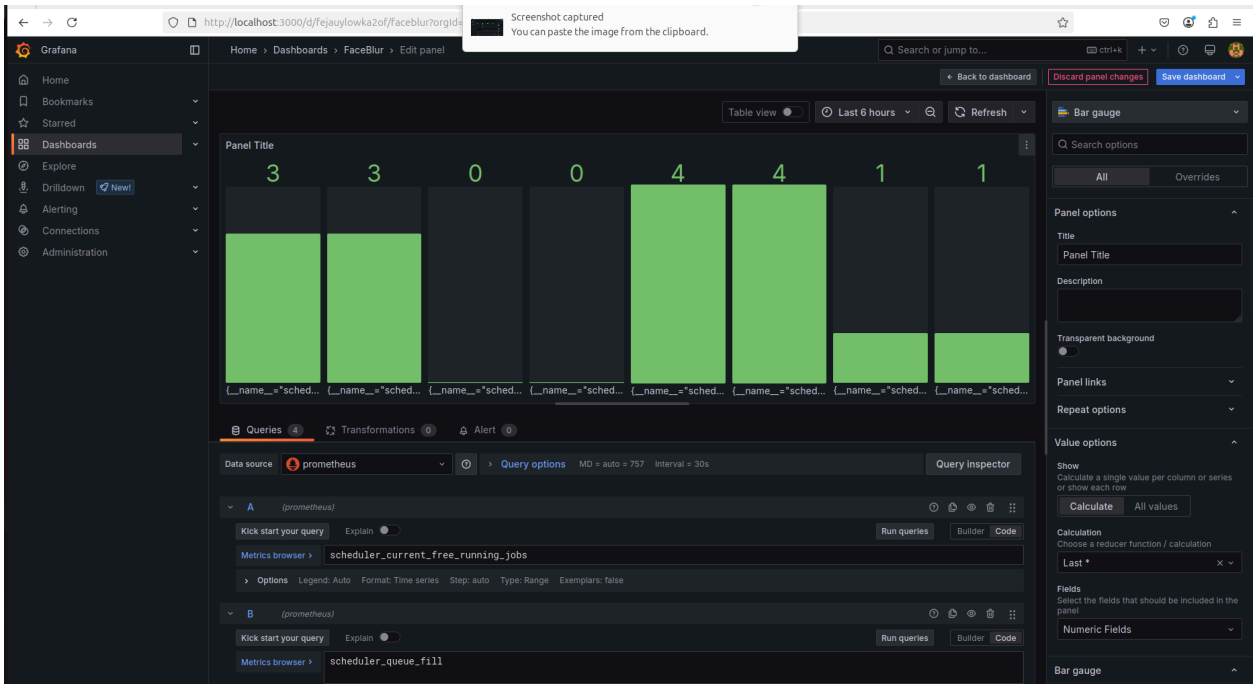

http://localhost:3000/d/fejauylowka2of/faceblur?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1
Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
ctrl+k
Back to dashboard
Discard panel changes
Save dashboard
Table view
Last 6 hours
Refresh
Panel Title
Histogram
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Histogram
Bucket count
approx bucket count
Default: 30
Bucket size
Auto
Bucket offset
for non-zero-based buckets
Default: 0
Combine series
combine all series into a single histogram
Stacking
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_queue_fill", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_queue_fill", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill


http://localhost:3000/d/fejauylowka2of/faceblur?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1
Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
ctrl+k
Back to dashboard
Discard panel changes
Save dashboard
Table view
Last 6 hours
Refresh
Panel Title
3
3
0
0
4
4
1
1
Stat
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Value options
Show
Calculate a single value per column or series or show each row
Calculate
All values
Calculation
Choose a reducer function / calculation
Last *
Fields
Select the fields that should be included in the panel
Numeric Fields
Stat styles
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill


http://localhost:3000/d/fejauylowka2of/faceblur?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1
Grafana
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
Back to dashboard
Discard panel changes
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Table view
Last 6 hours
Refresh
Panel Title
Pie chart
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Value options
Show
Calculate a single value per column or series or show each row
Calculate
All values
Calculation
Choose a reducer function / calculation
Last *
Fields
Select the fields that should be included in the panel
Numeric Fields
Pie chart
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_queue_fill", instance="localhost:18080", job="scheduler"}
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill


http://localhost:3000/d/fejauylowka2of/faceblur?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1
Grafana
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
Back to dashboard
Discard panel changes
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Table view
Last 6 hours
Refresh
Panel Title
Bar chart
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Bar chart
X Axis
First string or time field
Orientation
Auto
Horizontal
Vertical
Rotate x-axis tick labels
-90° -45° 0° 45° 90°
X-axis labels minimum spacing
None
Small
Medium
Large
Show values
Auto
Always
Never
01:32:30
01:33:00
01:33:30
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_free_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_queue_fill", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_queue_fill", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_queue_free", instance="localhost:18080", job="scheduler_metrics"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler"}
{__name__="scheduler_current_running_jobs", instance="localhost:18080", job="scheduler_metrics"}
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill


http://localhost:3000/d/fejauylowka2of/faceblur?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1
Grafana
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
Back to dashboard
Discard panel changes
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Table view
Last 6 hours
Refresh
Stat
Panel Title
3
3
0
0
4
4
1
1
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Value options
Show
Calculate a single value per column or series or show each row
Calculate
All values
Calculation
Choose a reducer function / calculation
Last *
Fields
Select the fields that should be included in the panel
Numeric Fields
Stat styles
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill


Screenshot captured
You can paste the image from the clipboard.
Grafana
Home › Dashboards › FaceBlur › Edit panel
Search or jump to...
Back to dashboard
Discard panel changes
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Table view
Last 6 hours
Refresh
Bar gauge
Panel Title
3
3
0
0
4
4
1
1
{__name__="sched...
Search options
All
Overrides
Panel options
Title
Panel Title
Description
Transparent background
Panel links
Repeat options
Value options
Show
Calculate a single value per column or series or show each row
Calculate
All values
Calculation
Choose a reducer function / calculation
Last *
Fields
Select the fields that should be included in the panel
Numeric Fields
Bar gauge
Queries 4
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 757
Interval = 30s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_current_free_running_jobs
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
scheduler_queue_fill

CHALLENGES

1. Initial Environment Setup Complexity

- **Multiple Dependencies**: Setting up the P2PFaaS environment required cloning and configuring multiple repositories (stack, stack-scheduler, stack-discovery, etc.), which had different environmental requirements.

- **Manual Environment Variable Configuration**: Environment variables for scheduler and discovery had to be manually exported and carefully matched with Docker and host configurations, increasing room for error.

2. Service Integration and Networking Issues

- **Inter-Service Communication**: Ensuring smooth communication between Scheduler, Discovery, and OpenFaaS services was challenging due to internal Docker networking and host IP mismatches.

- **Prometheus Target Reachability**: Prometheus often failed to scrape metrics initially due to incorrect port bindings or Prometheus not being able to reach the Scheduler's metrics endpoint (localhost:18080 within container context).

3. Prometheus Configuration Challenges

- **Config File Confusion**: Editing prometheus.yml required careful attention to syntax and correct formatting. Even small indentations or misalignment of job names and targets resulted in startup errors.

- **Debugging Unreachable Targets**: Diagnosing why Prometheus targets were marked as "down" required debugging both Prometheus logs and the Scheduler's logs concurrently.

4. Metrics Implementation in Go

- **Codebase Familiarity**: The Scheduler service was a pre-existing Go project. Understanding its structure (metrics.go, records.go, services.go) and inserting the appropriate Prometheus instrumentation required a learning curve.

- **Enabling and Validating Metrics**: Metrics were initially commented out; enabling them correctly and ensuring they actually logged meaningful data involved multiple iterations and print-debugging.

5. Grafana Setup and Configuration

- **Package Installation Delays**: Installing Grafana on a Debian-based system required additional repository configuration and sometimes failed due to missing dependencies or outdated apt keys.

- **Dashboard Configuration**: Creating meaningful Grafana dashboards required a basic understanding of PromQL queries, and mapping them to the right metrics with appropriate time ranges and visual types.

6. Validation and Testing Bottlenecks

- **Metric Visibility Delay**: Even after correct implementation, some metrics took time to show up on Prometheus/Grafana dashboards, leading to confusion about whether the system was working.

- **Debugging Without Errors**: When metrics didn't appear, there were often no errors—requiring careful step-by-step verification of endpoints, logs, and dashboards.

7. Scheduler and Function Invocation Testing

- **Pigo Faceblur Integration**: Invoking the pigo-faceblur function via OpenFaaS from the Scheduler required handling both the function deployment and testing through CLI/HTTP. Diagnosing delays or

non-200 status codes took time.

- **Response Delay Measurements**: Ensuring the Scheduler measured and logged average delay for function calls required synchronized testing across benchmark scripts and the logging system.

CONCLUSION

The successful implementation of the P2PFaaS framework represents a significant advancement in Function-as-a-Service (FaaS) systems for distributed edge and fog computing. Through this project, we explored the core components of the P2PFaaS stack, configured them for peer-to-peer operation, and tested various scheduling algorithms like NoScheduler, PowerOfNScheduler, and RoundRobinScheduler, gaining insights into their effects on system behavior, latency, and resource use.

The integration of Prometheus for real-time metrics and Grafana for visualization enhanced the system's monitoring and observability. This setup allowed us to track performance, detect anomalies, and analyze workload patterns—crucial for production-grade environments.

The inclusion of the Pigo FaceBlur function provided a real-world test case, demonstrating the platform's robustness for applications in edge intelligence and smart cities. This project has strengthened our understanding of edge computing, serverless systems, and monitoring, offering numerous opportunities for future research and improvements.

Overall, the project provides a solid foundation for scaling and further development, bridging theory and hands-on experience in distributed computing.

FUTURE WORK

Future work on the P2PFaaS framework can focus on several key enhancements to improve its scalability, intelligence, and usability. Advancing the metrics and monitoring layer by capturing fine-grained, per-function metrics and integrating alerting tools like Prometheus AlertManager would enhance observability and reliability. Introducing dynamic scheduler adaptation, such as hot-swapping policies at runtime and enabling context-aware task placement, would allow more responsive load balancing. Incorporating machine learning approaches—including predictive load forecasting and reinforcement learning-based scheduling—can make the system more proactive and adaptive. Expanding support for heterogeneous edge devices like Raspberry Pi or Jetson Nano, along with fault tolerance mechanisms such as automatic peer recovery and gossip-based state propagation, will make the system more robust in real-world deployments. Enhancing security through encrypted communication and access controls, along with building user-friendly dashboards and extending the OpenFaaS UI for better control and visibility, will improve system manageability. Finally, deploying the framework across geographically distributed fog nodes, benchmarking it under realistic workloads, and integrating CI/CD pipelines will ensure its readiness for industrial-grade use and ongoing development.